

Aula 1: Estruturas de Decisão em Java

As estruturas de decisão são fundamentais em qualquer linguagem de programação, permitindo que o fluxo de execução de um programa seja alterado com base em condições. Em Java, as principais estruturas de decisão são `if`, `if-else`, `if-else-if` e `switch`.

1. Estrutura `if`

A estrutura `if` executa um bloco de código se uma condição especificada for verdadeira.

Sintaxe

```
if (condicao) {  
    // Bloco de código a ser executado se a condição for verdadeira  
}
```

Exemplo Prático

```
public class ExemploIf {  
    public static void main(String[] args) {  
        int idade = 20;  
  
        if (idade >= 18) {  
            System.out.println("Você é maior de idade.");  
        }  
    }  
}
```

Exercício Resolvido

Problema: Verifique se um número é positivo.

Solução:

```
public class NumeroPositivo {  
    public static void main(String[] args) {  
        int numero = 10;  
  
        if (numero > 0) {  
            System.out.println("O número " + numero + " é positivo.");  
        }  
    }  
}
```

2. Estrutura `if-else`

A estrutura `if-else` executa um bloco de código se a condição for verdadeira e outro bloco se a condição for falsa.

Sintaxe

```
if (condicao) {  
    // Bloco de código se a condição for verdadeira  
} else {  
    // Bloco de código se a condição for falsa  
}
```

Exemplo Prático

```
public class ExemploIfElse {  
    public static void main(String[] args) {  
        int hora = 14;  
  
        if (hora < 12) {  
            System.out.println("Bom dia!");  
        } else {  
            System.out.println("Boa tarde!");  
        }  
    }  
}
```

Exercício Resolvido

Problema: Verifique se um número é par ou ímpar.

Solução:

```
public class ParOuImpar {  
    public static void main(String[] args) {  
        int numero = 7;  
  
        if (numero % 2 == 0) {  
            System.out.println("O número " + numero + " é par.");  
        } else {  
            System.out.println("O número " + numero + " é ímpar.");  
        }  
    }  
}
```

3. Estrutura if-else-if

A estrutura `if-else-if` permite testar múltiplas condições em sequência.

Sintaxe

```
if (condicao1) {  
    // Bloco de código se condicao1 for verdadeira  
} else if (condicao2) {  
    // Bloco de código se condicao2 for verdadeira  
} else {  
    // Bloco de código se nenhuma das condições anteriores for verdadeira  
}
```

Exemplo Prático

```
public class ExemploIfElseIf {  
    public static void main(String[] args) {  
        int nota = 75;  
  
        if (nota >= 90) {  
            System.out.println("Conceito A");  
        } else if (nota >= 80) {  
            System.out.println("Conceito B");  
        } else if (nota >= 70) {  
            System.out.println("Conceito C");  
        } else {  
            System.out.println("Conceito D");  
        }  
    }  
}
```

Exercício Resolvido

Problema: Classifique um triângulo com base nos comprimentos de seus lados (equilátero, isósceles, escaleno).

Solução:

```
public class TipoTriangulo {
    public static void main(String[] args) {
        int lado1 = 5;
        int lado2 = 5;
        int lado3 = 5;

        if (lado1 == lado2 && lado2 == lado3) {
            System.out.println("Triângulo Equilátero");
        } else if (lado1 == lado2 || lado1 == lado3 || lado2 == lado3) {
            System.out.println("Triângulo Isósceles");
        } else {
            System.out.println("Triângulo Escaleno");
        }
    }
}
```

4. Estrutura switch

A estrutura `switch` é usada para executar diferentes blocos de código com base em diferentes valores de uma única variável.

Sintaxe

```
switch (expressao) {
    case valor1:
        // Bloco de código a ser executado se expressao for igual a valor1
        break;
    case valor2:
        // Bloco de código a ser executado se expressao for igual a valor2
        break;
    default:
        // Bloco de código a ser executado se nenhum dos casos anteriores
        for correspondido
}
```

Exemplo Prático

```
public class ExemploSwitch {  
    public static void main(String[] args) {  
        int diaDaSemana = 3; // 1 = Domingo, 2 = Segunda, etc.  
  
        switch (diaDaSemana) {  
            case 1:  
                System.out.println("Domingo");  
                break;  
            case 2:  
                System.out.println("Segunda-feira");  
                break;  
            case 3:  
                System.out.println("Terça-feira");  
                break;  
            case 4:  
                System.out.println("Quarta-feira");  
                break;  
            case 5:  
                System.out.println("Quinta-feira");  
                break;  
            case 6:  
                System.out.println("Sexta-feira");  
                break;  
            case 7:  
                System.out.println("Sábado");  
                break;  
            default:  
                System.out.println("Dia inválido");  
        }  
    }  
}
```

Exercício Resolvido

Problema: Dada uma letra que representa um conceito (A, B, C, D, F), exiba uma mensagem correspondente.

Solução:

```
public class ConceitoSwitch {  
    public static void main(String[] args) {  
        char conceito = 'B';  
  
        switch (conceito) {  
            case 'A':  
                System.out.println("Excelente!");  
                break;  
            case 'B':  
                System.out.println("Muito Bom!");  
                break;  
            case 'C':  
                System.out.println("Bom!");  
                break;  
            case 'D':  
                System.out.println("Regular.");  
                break;  
            case 'F':  
                System.out.println("Reprovado.");  
                break;  
            default:  
                System.out.println("Conceito inválido.");  
        }  
    }  
}
```

Aula 2: Iteração e Laços de Repetição em Java

Os laços de repetição, também conhecidos como estruturas de iteração, permitem executar um bloco de código repetidamente enquanto uma condição específica for verdadeira ou por um número determinado de vezes. Em Java, os principais laços de repetição são `while`, `do-while` e `for`.

1. Laço while

O laço `while` executa um bloco de código repetidamente enquanto uma condição booleana for verdadeira. A condição é avaliada antes de cada iteração.

Sintaxe

```
while (condicao) {  
    // Bloco de código a ser executado repetidamente  
    // É importante que a condição eventualmente se torne falsa para evitar  
    um loop infinito  
}
```

Exemplo Prático

```
public class ExemploWhile {  
    public static void main(String[] args) {  
        int contador = 0;  
  
        while (contador < 5) {  
            System.out.println("Contador: " + contador);  
            contador++; // Incrementa o contador para que a condição  
eventualmente se torne falsa  
        }  
    }  
}
```

Exercício Resolvido

Problema: Calcule a soma dos números de 1 a 10 usando um laço `while`.

Solução:


```
public class SomaComWhile {  
    public static void main(String[] args) {  
        int numero = 1;  
        int soma = 0;  
  
        while (numero <= 10) {  
            soma += numero;  
            numero++;  
        }  
        System.out.println("A soma dos números de 1 a 10 é: " + soma);  
    }  
}
```

2. Laço do-while

O laço `do-while` é semelhante ao `while`, mas garante que o bloco de código seja executado pelo menos uma vez, pois a condição é avaliada após a execução do bloco.

Sintaxe

```
do {  
    // Bloco de código a ser executado repetidamente  
} while (condicao);
```

Exemplo Prático

```
import java.util.Scanner;

public class ExemploDowhile {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int numero;

        do {
            System.out.print("Digite um número positivo: ");
            numero = scanner.nextInt();
        } while (numero <= 0);

        System.out.println("Você digitou: " + numero);
        scanner.close();
    }
}
```

Exercício Resolvido

Problema: Peça ao usuário para digitar 's' ou 'n' até que uma das opções válidas seja inserida.

Solução:

```
import java.util.Scanner;

public class ValidarEntradaDowhile {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        char resposta;

        do {
            System.out.print("Deseja continuar? (s/n): ");
            resposta = scanner.next().charAt(0);
        } while (resposta != 's' && resposta != 'n');

        System.out.println("Você digitou: " + resposta);
        scanner.close();
    }
}
```

3. Laço for

O laço `for` é frequentemente usado quando o número de iterações é conhecido. Ele fornece uma sintaxe concisa para inicialização, condição e incremento/decremento.

Sintaxe

```
for (inicializacao; condicao; atualizacao) {
    // Bloco de código a ser executado repetidamente
}
```

Exemplo Prático

```
public class ExemploFor {  
    public static void main(String[] args) {  
        for (int i = 0; i < 5; i++) {  
            System.out.println("Iteração: " + i);  
        }  
    }  
}
```

Exercício Resolvido

Problema: Imprima os números pares de 0 a 10 usando um laço `for`.

Solução:

```
public class ParesComFor {  
    public static void main(String[] args) {  
        for (int i = 0; i <= 10; i += 2) {  
            System.out.println("Número par: " + i);  
        }  
    }  
}
```

4. Laço `for-each` (Enhanced for loop)

O laço `for-each` é uma forma simplificada de iterar sobre arrays e coleções. Ele torna o código mais legível e menos propenso a erros de índice.

Sintaxe

```
for (TipoElemento elemento : colecaoOuArray) {  
    // Bloco de código a ser executado para cada elemento  
}
```

Exemplo Prático

```
public class ExemploForEach {  
    public static void main(String[] args) {  
        String[] frutas = {"Maçã", "Banana", "Laranja"};  
  
        for (String fruta : frutas) {  
            System.out.println("Fruta: " + fruta);  
        }  
    }  
}
```

Exercício Resolvido

Problema: Calcule a soma dos elementos de um array de inteiros usando `for-each`.

Solução:

```
public class SomaArrayForEach {  
    public static void main(String[] args) {  
        int[] numeros = {10, 20, 30, 40, 50};  
        int soma = 0;  
  
        for (int num : numeros) {  
            soma += num;  
        }  
        System.out.println("A soma dos elementos do array é: " + soma);  
    }  
}
```

Aula 3: Laços Aninhados e Controle de Fluxo em Java

Além dos laços básicos (`while`, `do-while`, `for`), Java permite o uso de laços aninhados (um laço dentro de outro) e oferece instruções para controlar o fluxo de

execução dentro dos laços, como `break` e `continue`.

1. Laços Aninhados

Laços aninhados são estruturas onde um laço de repetição é colocado dentro de outro. Isso é útil para trabalhar com estruturas de dados bidimensionais, como matrizes, ou para gerar padrões.

Sintaxe (Exemplo com `for` aninhado)

```
for (inicializacao1; condicao1; atualizacao1) {  
    for (inicializacao2; condicao2; atualizacao2) {  
        // Bloco de código do laço interno  
    }  
    // Bloco de código do laço externo  
}
```

Exemplo Prático

```
public class ExemploLacosAninhados {  
    public static void main(String[] args) {  
        // Imprimir uma matriz 3x3  
        int[][] matriz = {  
            {1, 2, 3},  
            {4, 5, 6},  
            {7, 8, 9}  
        };  
  
        for (int i = 0; i < matriz.length; i++) { // Laço externo para  
linhas  
            for (int j = 0; j < matriz[i].length; j++) { // Laço interno  
para colunas  
                System.out.print(matriz[i][j] + " ");  
            }  
            System.out.println(); // Quebra de linha após cada linha da  
matriz  
        }  
    }  
}
```

Exercício Resolvido

Problema: Gere uma tabuada de multiplicação de 1 a 5 usando laços aninhados.

Solução:

```
public class TabuadaAninhada {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 5; i++) { // Multiplicando  
            System.out.println("Tabuada do " + i + ":");  
            for (int j = 1; j <= 10; j++) { // Multiplicador  
                System.out.println(i + " x " + j + " = " + (i * j));  
            }  
            System.out.println(); // Linha em branco para separar as  
tabuadas  
        }  
    }  
}
```

2. Instrução `break`

A instrução `break` é usada para sair imediatamente de um laço de repetição (ou de uma instrução `switch`). A execução do programa continua na instrução que segue o laço.

Exemplo Prático

```
public class ExemploBreak {  
    public static void main(String[] args) {  
        for (int i = 0; i < 10; i++) {  
            if (i == 5) {  
                System.out.println("Encontrou 5, saindo do laço.");  
                break; // Sai do laço for  
            }  
            System.out.println("Número: " + i);  
        }  
    }  
}
```

Exercício Resolvido

Problema: Procure por um número específico em um array e pare a busca assim que o encontrar.

Solução:


```
public class BuscarComBreak {  
    public static void main(String[] args) {  
        int[] numeros = {10, 25, 30, 45, 50};  
        int numeroProcurado = 30;  
        boolean encontrado = false;  
  
        for (int i = 0; i < numeros.length; i++) {  
            if (numeros[i] == numeroProcurado) {  
                System.out.println("Número " + numeroProcurado + "  
encontrado na posição " + i);  
                encontrado = true;  
                break;  
            }  
        }  
  
        if (!encontrado) {  
            System.out.println("Número " + numeroProcurado + " não  
encontrado no array.");  
        }  
    }  
}
```

3. Instrução continue

A instrução `continue` é usada para pular a iteração atual de um laço e prosseguir para a próxima iteração. O código restante dentro do bloco do laço para a iteração atual é ignorado.

Exemplo Prático

```
public class ExemploContinue {
    public static void main(String[] args) {
        for (int i = 0; i < 10; i++) {
            if (i % 2 != 0) { // Se o número for ímpar
                continue; // Pula para a próxima iteração
            }
            System.out.println("Número par: " + i);
        }
    }
}
```

Exercício Resolvido

Problema: Imprima todos os números de 1 a 10, exceto o número 5.

Solução:

```
public class PularNumeroComContinue {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                continue; // Pula a iteração quando i for 5
            }
            System.out.println("Número: " + i);
        }
    }
}
```